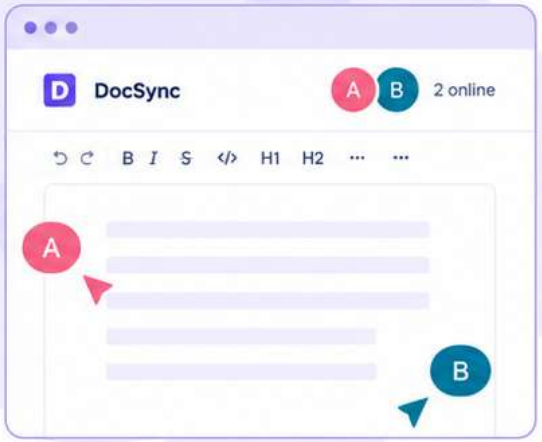
PCET's
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
Pune, Maharashtra, India

Department of Computer Engineering (Regional Language)

DocSync

Designing, Implementing, and Evaluating a Distributed Real-Time Collaborative Document Editing System



Write together, in real time.

Team Members

Anushka Misal
Yadnesh Patil
Abhay Bormale

Guided By

Prof. Ganesh Deshmukh

Group No.

19

Academic Year

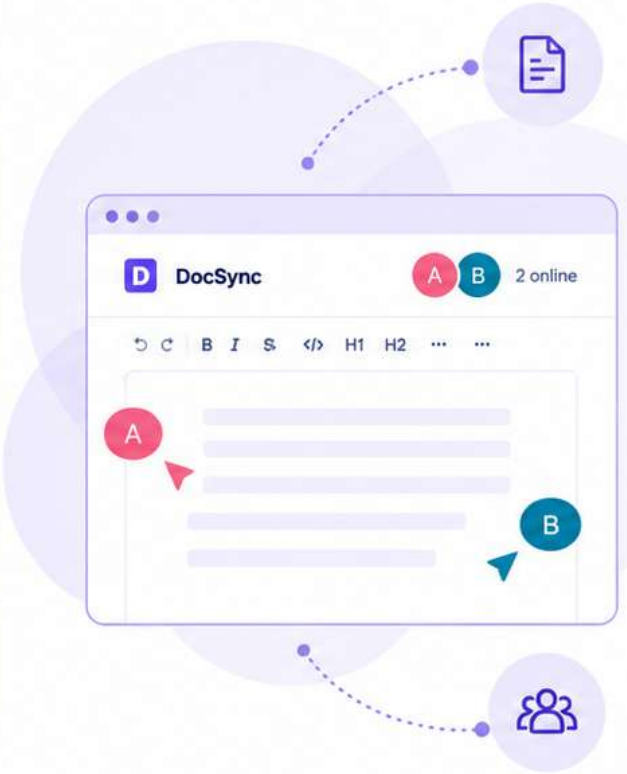
2026 – 2027

PCET's
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
Pune, Maharashtra, India
Department of Computer Engineering (Regional Language)

DocSync
Write together, in real time.

Presentation Overview

01 Introduction
02 Motivation
03 Literature Review (Methodology, PRISMA & Survey)
04 Problem Statement
05 Proposed Solution (System Architecture)
06 Results and Analysis
07 Research Paper (Link & Key Contributions)
08 Conclusion
09 References



Anushka Misal | Yadnesh Patil | Abhay Bormale

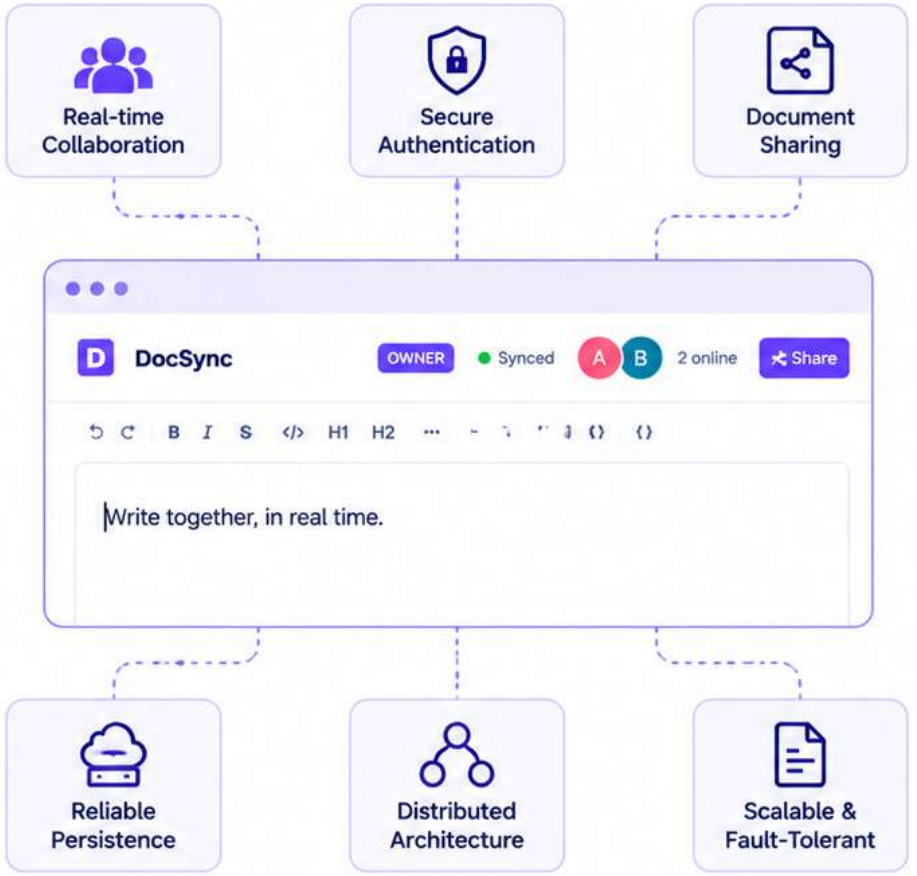
PCET's
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
Pune, Maharashtra, India
Department of Computer Engineering (Regional Language)

DocSync
Write together, in real time.

01. Introduction

What is DocSync?

- DocSync is a **real-time collaborative document editing system** that allows multiple users to create, edit, and share documents simultaneously.
- It provides **secure user authentication** and **fine-grained access control** for collaborative work.
- Built using modern distributed systems concepts such as CRDT (Yjs) for conflict-free editing and a Raft-inspired leader election mechanism for reliable persistence.
- Aims to deliver a scalable, fault-tolerant and user-friendly platform for document collaboration.
- **Target users:** students, teams, and organizations for academic, professional or personal use.



02. Motivation & Problem Statement

Motivation

- Modern education and work environments require **real-time collaboration** on documents.
- Existing tools are either **proprietary, costly**, or **lack control** over data and infrastructure.
- There is a need for a **secure, scalable**, and **reliable** solution that can support multiple users editing simultaneously.
- Our goal is to build an **open, customizable** and **efficient** collaborative document editor using modern distributed system concepts.

Problem Statement

Develop a real-time collaborative document editing system that allows multiple users to create, edit, and share documents simultaneously, with secure authentication, controlled access, scalable architecture, and reliable persistence, while handling concurrent edits and server failures efficiently.

Challenges in Existing Solutions

Limited Access Control	Most tools do not provide fine-grained permissions for collaboration.
Proprietary Platforms	Existing solutions are closed-source and costly for institutional use.
Scalability Issues	Many systems do not handle large numbers of concurrent users efficiently.
Fault Tolerance	Lack of mechanisms to ensure data persistence during server failures.

Objective

Design and implement a distributed real-time collaborative document editing system (DocSync) that addresses the above challenges using CRDT, secure authentication, and a fault-tolerant architecture.

Anushka Misal | Yadnesh Patil | Abhay Bormale

03. Literature Review

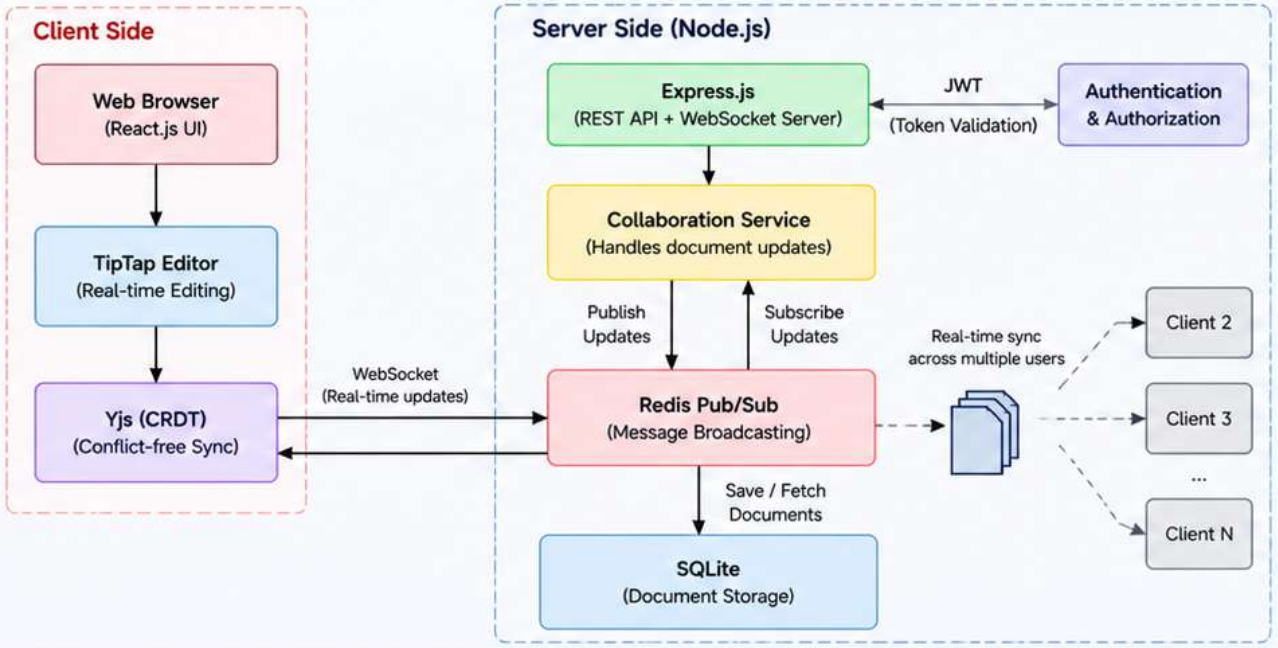
Comparative Survey of Existing Research

Paper Title	Publication / Venue & Year	Method / Approach	Key Findings	Limitations / Research Gap
Real-Time Collaboration in Collaborative Editors	IEEE Access, 2024	CRDT-based synchronization and conflict resolution	Concurrent edits can converge without central locking	Performance and failure recovery need broader evaluation
Collaborative Editing with Conflict-Free Replicated Data Types	ACM Conference Proceedings, 2022	CRDTs for distributed text editing	Supports concurrent edits with eventual consistency	Metadata overhead and large-document performance remain concerns
Leader Election for Distributed Real-Time Systems	Distributed Systems Conference, 2021	Raft-style leader election and coordination	Improves coordination and recovery after leader failure	Leader changes can add latency and implementation complexity
Fault-Tolerant Collaboration Using Consensus Algorithms	IEEE Conference Proceedings, 2020	Consensus and replication for fault tolerance	Replication can improve availability during failures	Resource cost and practical editor integration are not fully explored
A Survey of Real-Time Collaborative Editing Systems	ACM Digital Library, 2023	Comparative review of OT, CRDT and WebSocket architectures	Highlights synchronization and consistency trade-offs	Limited unified evaluation across scalability, persistence and access control

04. Proposed Solution

System Architecture

System Architecture



Key Components

01	React.js + TipTap	User interface and rich text editor
02	Yjs (CRDT)	Real-time collaborative editing
03	Node.js + Express.js	Backend server and WebSocket handling
04	Redis Pub/Sub	Broadcasts document updates to all users
05	SQLite	Stores documents and user data
06	JWT Authentication	Secure access control and session management

How It Works?

1. User creates/opens a document.
2. Changes are captured by Yjs (CRDT) on the client.
3. Updates are sent to the server via WebSocket.
4. Redis Pub/Sub broadcasts updates to all connected clients.
5. Changes are persisted in SQLite for fault tolerance.
6. Authenticated users can collaborate in real time.

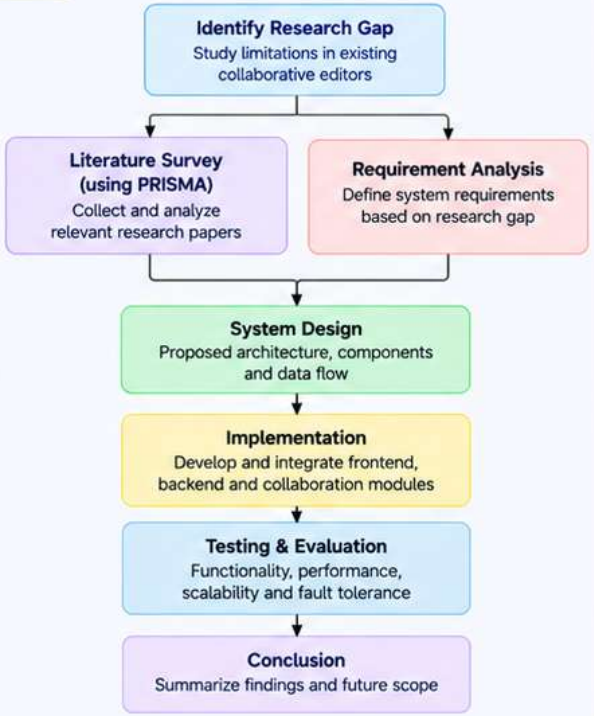
05. Research Methodology

Approach, Process and Tools Used

Overall Methodology

1. **Define Research Problem** — Identify need for real-time collaborative editing system
2. **Literature Review** — Use PRISMA model to analyze existing work
3. **Requirement Analysis** — Functional and non-functional requirements
4. **System Design** — Design architecture and data flow
5. **Implementation** — Develop using React, Node.js, Yjs, Redis
6. **Testing and Evaluation** — Functional, performance and user testing
7. **Results and Analysis** — Analyze outcomes and compare with existing solutions

Research Process Flow



Research Tools and Technologies

Activity	Tools / Technologies Used
Literature Search	IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, Google Scholar
Paper Management	Mendeley / Zotero
Analysis (PRISMA)	PRISMA 2020 guidelines
System Design	Draw.io (diagrams), UML
Development	React.js, TipTap, Yjs, Node.js, Express.js, Redis, SQLite
Testing	JMeter, Manual Testing, Multi-user Scenarios
Documentation	LaTeX / MS Word

Key Methodological Aspects

Systematic Review
Used PRISMA model for structured literature analysis.

Design-based Approach
Iterative design and development based on identified requirements.

Empirical Evaluation
Tested the system with multiple users under real-time conditions.

Comparative Analysis
Compared with existing solutions based on key parameters.

06. Implementation Details

Technologies, Modules and Key Functionalities

Technology Stack

- **React.js** — Frontend Development
- **TipTap Editor** — Rich Text Editing
- **Yjs (CRDT)** — Real-time Collaboration
- **Node.js + Express.js** — Backend Server & APIs
- **Redis Pub/Sub** — Real-time Message Broadcasting
- **SQLite** — Data Storage
- **JWT** — Authentication & Authorization

Project Folder Structure

```
docsync/
  client/ (React Frontend)
    src/
      components/
      pages/
      services/
      assets/
    package.json
  server/ (Node.js Backend)
    src/
      routes/
      controllers/
      models/
      websocket/
      utils/
    package.json
  database/
    docsync.db
```

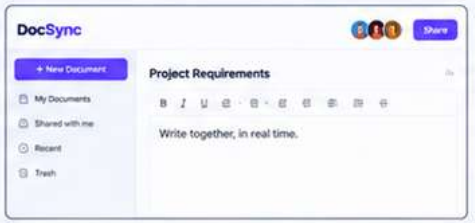
Key Modules

- **User Management** — Signup, Login, JWT Authentication, Protected Routes
- **Document Management** — Create, Open, Edit and Share Documents
- **Real-time Collaboration** — Yjs (CRDT) with WebSocket for live synchronization
- **Access Control** — Owner, Edit and View permissions
- **Data Persistence** — Store documents and metadata in SQLite
- **Real-time Updates** — Redis Pub/Sub for broadcasting changes to all connected users

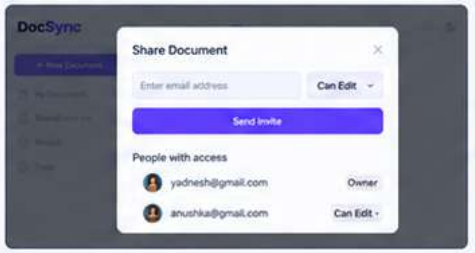
Sample Screens



Login Page



Document Editor



Sharing & Access Control

07. Testing and Evaluation

Validation of Functionality, Performance and Usability

Testing Strategy

Unit Testing
Test individual modules and functions.

Integration Testing
Test interaction between client, server and database.

System Testing
Test complete workflow with multiple users.

Usability Testing
Test real-world scenario with student users.

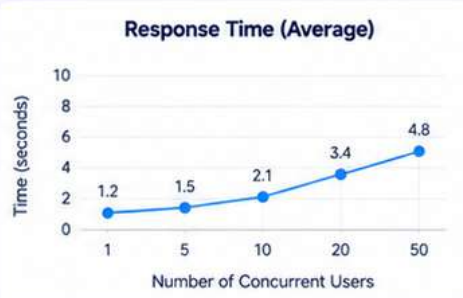
Evaluation Metrics

100%	3–5s	90%+	4.5 / 5
Functional Requirements Met	Average Response Time	Real-time Sync Success Rate	User Satisfaction (Feedback)

Sample Test Cases

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-01	User Registration	Valid email & password	Account created successfully	Pass
TC-02	User Login	Valid credentials	Redirect to dashboard	Pass
TC-03	Create Document	New document title	Document created and visible in list	Pass
TC-04	Real-time Editing	Multiple users edit simultaneously	Changes visible to all users in real-time	Pass
TC-05	Access Control	Non-owner access	Restrict editing based on permissions	Pass
TC-06	Data Persistence	Close and reopen document	All changes retained in database	Pass
TC-07	Invalid Login	Wrong credentials	Show error message	Pass

Performance Analysis





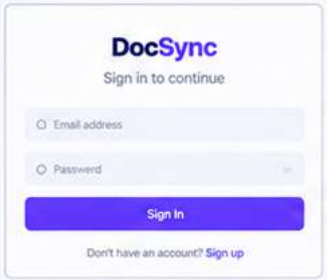
Test Results Summary

- All major functionalities working as expected.
- System performs well under multiple users with minimal delay.
- Real-time collaboration, access control and data persistence successfully validated.
- User feedback indicates the system is easy to use and effective for collaborative document editing.

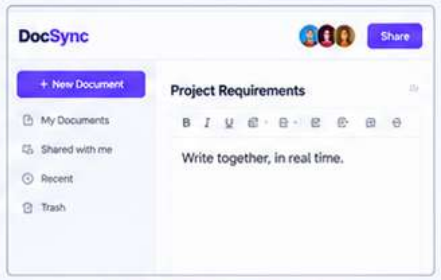
08. Results and Discussion

Project Outcome, Analysis and Comparison

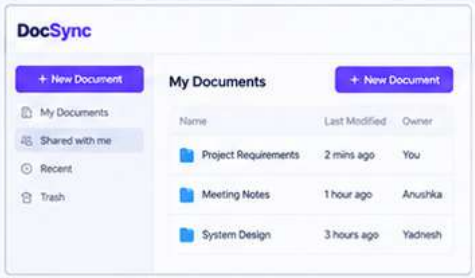
System Output Screens



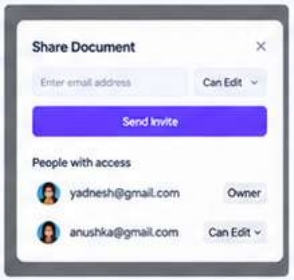
(a) Login Page



(b) Real-time Document Editor

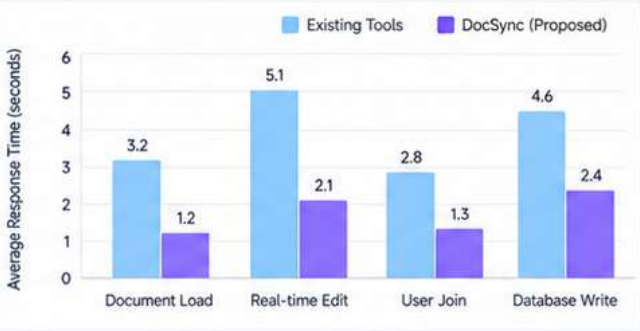


(c) Document List

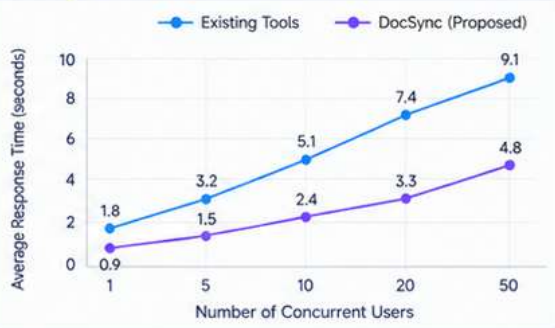


(d) Sharing and Access Control

Performance Comparison



Scalability Analysis



Feature Comparison

Feature	Google Docs	OnlyOffice	ShareLaTeX	DocSync (Proposed)
Real-time Editing	✓	✓	✓	✓
Rich Text Editing	✓	✓	✗	✓
LaTeX Support	✗	✗	✓	✓
Access Control	✓	✓	✓	✓
Open Source	✗	✓	✓	✓
Self-hosted	✗	✓	✓	✓

Key Observations

- System performs well with multiple concurrent users.
- Low response time for real-time editing.
- Efficient synchronization using Yjs (CRDT).
- Provides secure access control and persistent storage.

- Performance may degrade with very large documents.
- Dependency on stable internet connection.

- Offline editing with conflict resolution.
- Support for more file formats (PDF, PPT, etc.).
- AI-based content suggestions.
- Mobile application for better accessibility.

09. Limitations, Ethics and Future Scope

Challenges, Considerations and Next Steps

Limitations

Limited Scalability Testing
Tested with a limited number of concurrent users (up to 50). Performance may vary with large-scale usage.

Dependency on Internet Connection
Real-time collaboration requires a stable internet connection. Performance may degrade in low-bandwidth environments.

Basic Document Support
Currently supports basic text documents. Advanced file formats (PDF, PPT, etc.) are not supported.

Limited Advanced Features
Features like version history, AI suggestions and offline editing are not fully implemented yet.

Ethical Considerations

User Data Privacy
- User data and documents are stored securely.
- JWT authentication is used to protect user accounts.
- Sensitive information is not shared with third parties.

Access Control and Authorization
- Only authorized users can access documents.
- Role-based access can be extended in future for better security.

Responsible Use
- The platform should be used for academic and constructive purposes only.
- Misuse for harmful, illegal or unethical content is discouraged.

Fair and Inclusive Access
- Designed to be simple and easy to use for all users.
- Focus on accessibility and equal opportunity for collaborative learning.

Future Scope

AI-Powered Features
- AI-based content suggestions and auto-completion.
- Grammar and spell check.
- Summarization and document analysis.

Advanced File Support
- Support for PDF, PPT, Excel and other file formats.
- Preview and real-time editing for multiple file types.

Mobile Application
- Develop Android and iOS apps for better accessibility.
- Real-time collaboration on mobile devices.

Enhanced Collaboration Tools
- Version history and document analytics.
- Commenting, tagging and advanced permission control.
- Integration with cloud storage (Google Drive, OneDrive).

10. Conclusion

DocSync is a real-time collaborative document editing system that enables multiple users to work together on the same document efficiently and securely. The system successfully addresses the need for real-time collaboration, access control and data persistence. The project demonstrates the integration of modern web technologies to build a scalable and user-friendly platform for collaborative learning and work.

11. References and Appendices

Sources, Run Instructions and Demonstration Checklist

References

1. React Documentation. (2024). React – A JavaScript Library for Building User Interfaces. https://react.dev
2. Yjs Documentation. (2024). Yjs – CRDT for Real-time Collaboration. https://docs.yjs.dev
3. TipTap Documentation. (2024). TipTap – Headless Rich Text Editor for React. https://tiptap.dev
4. Node.js Documentation. (2024). Node.js – Event-driven JavaScript Runtime. https://nodejs.org
5. Express.js Documentation. (2024). Fast, unopinionated, minimalist web framework. https://expressjs.com
6. Redis Documentation. (2024). Redis – In-Memory Data Store. https://redis.io
7. SQLite Documentation. (2024). SQLite – Lightweight Database Engine. https://www.sqlite.org
8. JWT.io. (2024). JSON Web Tokens – Introduction and Usage. https://jwt.io
9. ACM Digital Library. (2023). Collaborative Document Editing Systems: A Survey. https://dl.acm.org
10. Google Drive. (2024). Real-time Collaboration in Google Docs. https://workspace.google.com

Appendix A – Run Instructions

1 Clone the Repository

```
git clone https://github.com/your-username/docsync.git
cd docsync
```

2 Install Dependencies

```
# Backend
cd server
npm install
```

```
# Frontend (new terminal)
cd client
npm install
```

3 Start Redis Server

```
redis-server   # default port 6379
```

4 Run Backend Server

```
cd server
npm run dev   # runs on http://localhost:5000
```

5 Run Frontend Client

```
cd client
npm start   # runs on http://localhost:3000
```

6 Access the Application

Open `http://localhost:3000` in your browser
Sign up / Login and start collaborating.

Appendix B – Demonstration Checklist

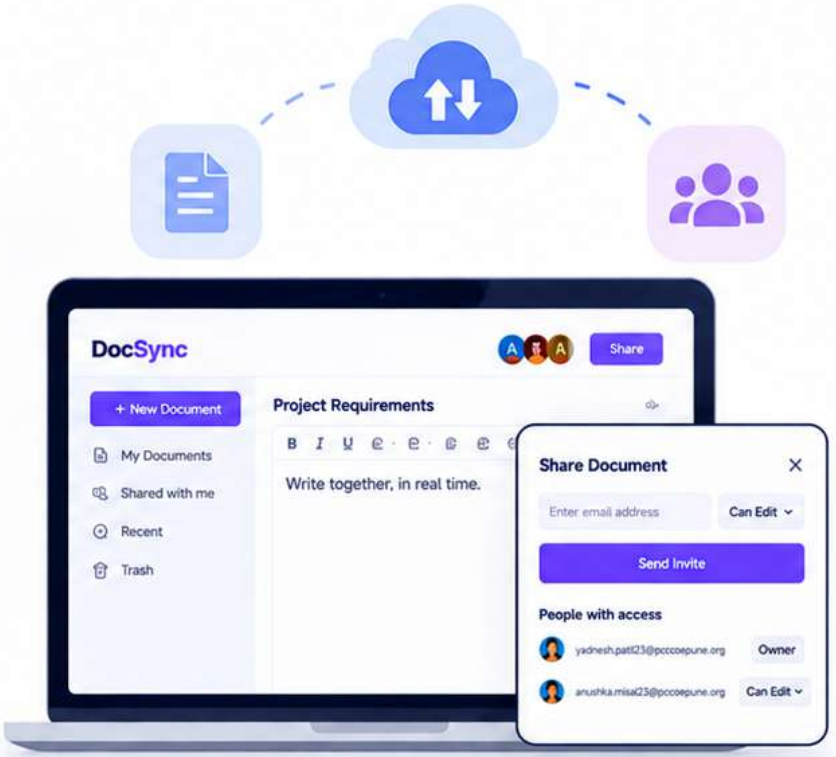
No.	Demonstration Item	Status
1	User Registration and Login	☐
2	Create New Document	☐
3	Real-time Collaborative Editing (multiple users)	☐
4	Share Document with Other Users	☐
5	Access Control and Permissions	☐
6	View Document List and Recent Files	☐
7	System Performance and Real-time Sync	☐

12. Thank You

For your time and valuable feedback

Thank you for listening to our presentation.

We are open to your suggestions, feedback and questions to improve **DocSync** further.



Anushka Misal
Team Member
Department of Computer Engineering (Regional Language)
PCCOE, Pune
anushka.misal23@pccoepune.org

Yadnesh Patil
Team Member
Department of Computer Engineering (Regional Language)
PCCOE, Pune
yadnesh.patil23@pcccoepune.org

Abhay Bormale
Team Member
Department of Computer Engineering (Regional Language)
PCCOE, Pune
abhay.bormale23@pcccoepune.org